



JAIN
DEEMED-TO-BE UNIVERSITY

SCHOOL OF
COMPUTER
SCIENCE AND IT

DEPARTMENT OF CS & IT

BCA – ITH – 4TH SEMESTER

ACADEMIC YEAR : 2024 – 2025 (EVEN SEMESTER)

BATCH : 2023 – 2026

TITLE	PYTHON PROGRAMMING
SUBJECT CODE	23BCA4C03
HOURS PER WEEK	3 Hrs/Week
CREDITS	3

MODULE 3

TOPICS TO BE COVERED**OBJECT ORIENTED PROGRAMMING AND REGULAR EXPRESSIONS**

1. Classes: Classes and Instances
2. Inheritance
3. Polymorphism
4. Abstract Classes
5. Exceptional Handling
6. Regular Expressions using “re” module.

OBJECT ORIENTED PROGRAMMING AND REGULAR EXPRESSIONS**1. CLASSES: CLASSES AND INSTANCES**

A class in Python is a user-defined template for creating objects. It bundles data and functions together, making it easier to manage and use them. When we create a new

class, we define a new type of object. We can then create multiple instances of this object type.

Classes are created using **class keyword**. Attributes are variables defined inside the class and represent the properties of the class. Attributes can be accessed using the dot **. operator** (e.g., MyClass.my_attribute).

Create a Class

```
# define a class
class Dog:
    sound = "bark" # class attribute
```

Create Object

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data.

Now, let's create an object from **Dog class**.

```
class Dog:
    sound = "bark"
# Create an object from the class
dog1 = Dog()
# Access the class attribute
print(dog1.sound)
```

sound attribute is a class attribute. It is shared across all instances of Dog class, so can be directly accessed through instance **dog1**.

Using __init__() Function

In Python, class has [__init__\(\)](#) function. It automatically initializes object attributes when an object is created.

```
class Dog:
    species = "Canine" # Class attribute
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute
```

Explanation:

- **class Dog:** Defines a class named Dog.
- **species:** A class attribute shared by all instances of the class.
- **__init__ method:** Initializes the name and age attributes when a new object is created.

Initiate Object with __init__

```
class Dog:
    species = "Canine" # Class attribute
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute
# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)
print(dog1.name) # Output: Buddy
print(dog1.species) # Output: Canine
```

Output

```
Buddy
Canine
```

Explanation:

- **dog1 = Dog("Buddy", 3):** Creates an object of the Dog class with name as "Buddy" and age as 3.
- **dog1.name:** Accesses the instance attribute name of the dog1 object.
- **dog1.species:** Accesses the class attribute species of the dog1 object.

Self Parameter

[self](#) parameter is a reference to the current instance of the class. It allows us to access the attributes and methods of the object.

```
class Dog:
    def bark(self):
        print(self.name)
dog1 = Dog("Buddy", 3)
dog1.bark()
```

Explanation:

- **self.name:** Refers to the name attribute of the object (dog1) calling the method.
- **dog1.bark():** Calls the bark method on dog1.

__str__ Method

`__str__` method in Python allows us to define a custom string representation of an object. By default, when we print an object or convert it to a string using `str()`, Python uses the default implementation, which returns a string like `<__main__.ClassName object at 0x00000123>`.

```
class Dog:
    def __init__(self, name, age):
```

```
        self.name = name
        self.age = age
    def __str__(self):
        return f"{self.name} is {self.age} years old." # Correct: Returning a string
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)
print(dog1)
print(dog2)
```

Output

```
Buddy is 3 years old.
Charlie is 5 years old.
```

Explanation:

- **__str__ Implementation:** Defined as a method in the Dog class. Uses the self parameter to access the instance's attributes (name and age).
- **Readable Output:** When print(dog1) is called, Python automatically uses the __str__ method to get a string representation of the object. Without __str__, calling print(dog1) would produce something like <__main__.Dog object at 0x00000123>.

Class and Instance Variables in Python

In Python, variables defined in a class can be either [class variables](#) or instance variables, and understanding the distinction between them is crucial for object-oriented programming.

Class Variables

These are the variables that are shared across all instances of a class. It is defined at the class level, outside any methods. All objects of the class share the same value for a class variable unless explicitly overridden in an object.

Instance Variables

Variables that are unique to each instance (object) of a class. These are defined within __init__ method or other instance methods. Each object maintains its own copy of instance variables, independent of other objects.

Example:

```
class Dog:
    # Class variable
    species = "Canine"
    def __init__(self, name, age):
```

```
# Instance variables
self.name = name
self.age = age

# Create objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

# Access class and instance variables
print(dog1.species) # (Class variable)
print(dog1.name)    # (Instance variable)
print(dog2.name)    # (Instance variable)

# Modify instance variables
dog1.name = "Max"
print(dog1.name)    # (Updated instance variable)

# Modify class variable
Dog.species = "Feline"
print(dog1.species) # (Updated class variable)
print(dog2.species)
```

Output

```
Canine
Buddy
Charlie
Max
Feline
Feline
```

Explanation:

- **Class Variable (species):** Shared by all instances of the class. Changing `Dog.species` affects all objects, as it's a property of the class itself.
- **Instance Variables (name, age):** Defined in the `__init__` method. Unique to each instance (e.g., `dog1.name` and `dog2.name` are different).
- **Accessing Variables:** Class variables can be accessed via the class name (`Dog.species`) or an object (`dog1.species`). Instance variables are accessed via the object (`dog1.name`).
- **Updating Variables:** Changing `Dog.species` affects all instances. Changing `dog1.name` only affects `dog1` and does not impact `dog2`.

2. Inheritance

Inheritance is a fundamental concept in [object-oriented programming](#) (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class). This promotes code reuse, modularity, and a hierarchical class structure. In this article, we'll explore inheritance in [Python](#).

Basic Example of Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class.

```
# Parent class
class Animal:
    def __init__(self, name):
        self.name = name # Initialize the name attribute
    def speak(self):
        pass # Placeholder method to be overridden by child classes
# Child class inheriting from Animal
class Dog(Animal):
    def speak(self):
        return f"{self.name} barks!" # Override the speak method
# Creating an instance of Dog
dog = Dog("Buddy")
print(dog.speak()) # Output: Buddy says Woof!
```

Output

```
Buddy barks!
```

Explanation:

- Animal is the parent class with an `__init__` method and a `speak` method.
- Dog is the child class that inherits from Animal.
- The `speak` method is overridden in the Dog class to provide specific behavior.
- [Syntax for Inheritance](#)
- [Explanation of Python Inheritance Syntax](#)
- [Creating a Parent Class](#)
- [Creating a Child Class](#)
- [__init__\(\) Function](#)
- [super\(\) Function](#)
- [Add Properties](#)

- **Types of Python Inheritance**

Syntax for Inheritance

```
class ParentClass:
    # Parent class code here

pass

class ChildClass(ParentClass):
    # Child class code here

pass
```

Explanation of Python Inheritance Syntax**1. Parent Class:**

- This is the base class from which other classes inherit.
- It contains attributes and methods that the child class can reuse.

2. Child Class:

- This is the derived class that inherits from the parent class.
- The syntax for inheritance is class ChildClass(ParentClass).
- The child class automatically gets all attributes and methods of the parent class unless overridden.

Creating a Parent Class

In object-oriented programming, a parent [class](#) (also known as a base class) defines common attributes and methods that can be inherited by other classes. These attributes and methods serve as the foundation for the child classes. By using inheritance, child classes can access and extend the functionality provided by the parent class.

Here's an example where Person is the parent class:

```
# A Python program to demonstrate inheritance

class Person(object):
    # Constructor
    def __init__(self, name, id):
        self.name = name
        self.id = id
    # To check if this person is an employee
    def Display(self):
        print(self.name, self.id)

# Driver code
emp = Person("Satyam", 102) # An Object of Person
```

```
emp.Display()
```

Output

```
Satyam 102
```

Explanation:

- The Person class has two attributes: name and id. These are set when an [object](#) of the class is created.
- The display method prints the name and id of the person.

Creating a Child Class

A child class (also known as a subclass) is a class that inherits properties and methods from its parent class. The child class can also introduce additional attributes and methods, or even override the ones inherited from the parent.

In this case, Emp is the child class that inherits from the Person class:

```
class Emp(Person):
    def Print(self):
        print("Emp class called")
    Emp_details = Emp("Mayank", 103)
# calling parent class function
Emp_details.Display()
# Calling child class function
Emp_details.Print()
```

Explanation:

- **Emp class** inherits the name and id attributes and the display method from the Person class.
- **__init__ method** in Emp calls `super().__init__(name, id)` to invoke the constructor of the Person class and initialize the inherited attributes.
- **Emp** introduces an additional attribute, role, and also overrides the display method to print the role in addition to the name and id.

`__init__()` Function

[__init__\(\) function](#) is a constructor method in Python. It initializes the object's state when the object is created. If the child class does not define its own `__init__()` method, it will automatically inherit the one from the parent class.

In the example above, the `__init__()` method in the Employee class ensures that both inherited and new attributes are properly initialized.

```
# Parent Class: Person
```

```
class Person:
```

```
def __init__(self, name, idnumber):
    self.name = name
    self.idnumber = idnumber
# Child Class: Employee
class Employee(Person):
    def __init__(self, name, idnumber, salary, post):
        super().__init__(name, idnumber) # Calls Person's __init__()
        self.salary = salary
        self.post = post
```

Explanation:

- **__init__() method** in Person initializes name and idnumber.
- **__init__() method** in Employee calls **super().__init__(name, idnumber)** to initialize the name and idnumber inherited from the Person class and adds salary and post.

super() Function

[super\(\) function](#) is used to call the parent class's methods. In particular, it is commonly used in the child class's **__init__()** method to initialize inherited attributes. This way, the child class can leverage the functionality of the parent class.

Example:

```
# Parent Class: Person
class Person:
    def __init__(self, name, idnumber):
        self.name = name
        self.idnumber = idnumber
    def display(self):
        print(self.name)
        print(self.idnumber)
# Child Class: Employee
class Employee(Person):
    def __init__(self, name, idnumber, salary, post):
        super().__init__(name, idnumber) # Using super() to call Person's __init__()
        self.salary = salary
        self.post = post
```

Explanation:

- The `super()` function is used inside the `__init__()` method of `Employee` to call the constructor of `Person` and initialize the inherited attributes (name and idnumber).
- This ensures that the parent class functionality is reused without needing to rewrite the code in the child class.

Add Properties

Once inheritance is established, both the parent and child classes can have their own properties. Properties are attributes that belong to a class and are used to store data.

Example:

Parent Class: Person

class Person:

def __init__(self, name, idnumber):

 self.name = name

 self.idnumber = idnumber

def display(self):

 print(self.name)

 print(self.idnumber)

Child Class: Employee

class Employee(Person):

def __init__(self, name, idnumber, salary, post):

 super().__init__(name, idnumber)

 self.salary = salary

 self.post = post

Explanation:

- Person class has properties name and **idnumber**.
- Employee class adds properties salary and post.
- The properties are initialized when an object is created, and they represent the specific data related to the Person and Employee.

Types of Python Inheritance

1. **Single Inheritance:** A child class inherits from one parent class.
2. **Multiple Inheritance:** A child class inherits from more than one parent class.
3. **Multilevel Inheritance:** A class is derived from a class which is also derived from another class.
4. **Hierarchical Inheritance:** Multiple classes inherit from a single parent class.
5. **Hybrid Inheritance:** A combination of more than one type of inheritance.

Example:

1. Single Inheritance

class Person:

```
def __init__(self, name):
    self.name = name
```

class Employee(Person): *# Employee inherits from Person*

```
def __init__(self, name, salary):
    super().__init__(name)
    self.salary = salary
```

2. Multiple Inheritance

class Job:

```
def __init__(self, salary):
    self.salary = salary
```

class EmployeePersonJob(Employee, Job): *# Inherits from both Employee and Job*

```
def __init__(self, name, salary):
    Employee.__init__(self, name, salary) # Initialize Employee
    Job.__init__(self, salary) # Initialize Job
```

3. Multilevel Inheritance

class Manager(EmployeePersonJob): *# Inherits from EmployeePersonJob*

```
def __init__(self, name, salary, department):
    EmployeePersonJob.__init__(self, name, salary) # Explicitly initialize EmployeePersonJob
    self.department = department
```

4. Hierarchical Inheritance

class AssistantManager(EmployeePersonJob): *# Inherits from EmployeePersonJob*

```
def __init__(self, name, salary, team_size):
    EmployeePersonJob.__init__(self, name, salary) # Explicitly initialize EmployeePersonJob
    self.team_size = team_size
```

5. Hybrid Inheritance (Multiple + Multilevel)

class SeniorManager(Manager, AssistantManager): *# Inherits from both Manager and AssistantManager*

```
def __init__(self, name, salary, department, team_size):
    Manager.__init__(self, name, salary, department) # Initialize Manager
```

```
AssistantManager.__init__(self, name, salary, team_size)  # Initialize
AssistantManager
# Creating objects to show inheritance
# Single Inheritance
emp = Employee("John", 40000)
print(emp.name, emp.salary)
# Multiple Inheritance
emp2 = EmployeePersonJob("Alice", 50000)
print(emp2.name, emp2.salary)
# Multilevel Inheritance
mgr = Manager("Bob", 60000, "HR")
print(mgr.name, mgr.salary, mgr.department)
# Hierarchical Inheritance
asst_mgr = AssistantManager("Charlie", 45000, 10)
print(asst_mgr.name, asst_mgr.salary, asst_mgr.team_size)
# Hybrid Inheritance
sen_mgr = SeniorManager("David", 70000, "Finance", 20)
print(sen_mgr.name, sen_mgr.salary, sen_mgr.department, sen_mgr.team_size)
```

Output

```
John 40000
Alice 50000
Bob 60000 HR
Charlie 45000 10
David 70000 Finance 20
```

Explanation:

1. **Single Inheritance:** Employee inherits from Person, adding a salary attribute.
2. **Multiple Inheritance:** EmployeePersonJob inherits from both Employee and Job, allowing access to both name and salary.
3. **Multilevel Inheritance:** Manager inherits from EmployeePersonJob, which already includes Employee and Job.
4. **Hierarchical Inheritance:** AssistantManager also inherits from EmployeePersonJob, demonstrating multiple child classes inheriting from the same parent.

5. **Hybrid Inheritance:** SeniorManager inherits from both Manager (multilevel) and AssistantManager (hierarchical), combining two inheritance types.

3. Polymorphism

Polymorphism is a foundational concept in programming that allows entities like functions, methods or operators to behave differently based on the type of data they are handling. Derived from Greek, the term literally means “many forms”.

Python’s dynamic typing and duck typing make it inherently polymorphic. Functions, operators and even built-in objects like loops exhibit polymorphic behavior.

Polymorphism in Built-in Functions

[Python’s built-in functions](#) exhibit polymorphism, adapting to various data types.

Example:

```
print(len("Hello")) # String length
print(len([1, 2, 3])) # List length
print(max(1, 3, 2)) # Maximum of integers
print(max("a", "z", "m")) # Maximum in strings
```

Python determines behavior at runtime, enabling these functions to work across diverse types without explicit type declarations.

- [Polymorphism in Built-in Functions](#)
- [Polymorphism in Functions](#)
- [Polymorphism in Operators](#)
- [Polymorphism in Object-Oriented Programming \(OOP\)](#)
- [Types of Polymorphism](#)
- [Inheritance Class Polymorphism](#)

Polymorphism in Functions

[Duck typing](#) enables functions to work with any object regardless of its type.

Example:

```
def add(a, b):
    return a + b
print(add(3, 4)) # Integer addition
print(add("Hello, ", "World!")) # String concatenation
print(add([1, 2], [3, 4])) # List concatenation
```

Polymorphism in Operators

Operator Overloading

In Python, operators like + behave polymorphically, performing addition, [concatenation](#) or merging based on the [data type](#).

Example:

```
print(5 + 10) # Integer addition
print("Hello " + "World!") # String concatenation
print([1, 2] + [3, 4]) # List concatenation
```

Polymorphism in OOPs

In [OOP](#), polymorphism allows methods in different [classes](#) to share the same name but perform distinct tasks. This is achieved through inheritance and interface design. Polymorphism complements other OOP principles like [inheritance](#) (sharing behavior) and encapsulation (hiding complexity) to create robust and modular applications.

Example:

```
class Shape:
    def area(self):
        return "Undefined"

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

shapes = [Rectangle(2, 3), Circle(5)]
for shape in shapes:
    print(f"Area: {shape.area()}")
```

Explanation:

- The code showcases polymorphism using a parent class Shape and child classes Rectangle and Circle.
- **Parent Class Shape:** Contains a generic area method returning "Undefined", acting as a placeholder for derived classes to override.

- **Child Class Rectangle:** Initializes length and width via the `__init__` constructor. Overrides the area method to return the rectangle's area as `length * width`.
- **Child Class Circle:** Initializes radius via the `__init__` constructor. Overrides the area method to return the circle's area as `3.14 * radius^2`.
- **Polymorphic Behavior:** A list of shape objects (Rectangle and Circle) is created. A for loop iterates through the list, calling the area method on each object. The method executed is determined by the object's type, showcasing polymorphism.

Types of Polymorphism

Compile-time Polymorphism

- Found in statically typed languages like Java or C++, where the behavior of a function or operator is resolved during the program's compilation phase.
- Examples include [method overloading](#) and operator overloading, where multiple functions or operators can share the same name but perform different tasks based on the context.
- In Python, which is dynamically typed, compile-time polymorphism is not natively supported. Instead, Python uses techniques like dynamic typing and duck typing to achieve similar flexibility.

Runtime Polymorphism

- Occurs when the behavior of a method is determined at runtime based on the type of the object.
- In Python, this is achieved through [method overriding](#): a child class can redefine a method from its parent class to provide its own specific implementation.
- Python's dynamic nature allows it to excel at runtime polymorphism, enabling flexible and adaptable code.

Example:

```
class Animal:
    def sound(self):
        return "Some generic sound"
class Dog(Animal):
    def sound(self):
        return "Bark"
class Cat(Animal):
    def sound(self):
```

```
        return "Meow"
# Polymorphic behavior
animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound()) # Calls the overridden method based on the object type
```

Output

```
Bark
Meow
Some generic sound
```

Explanation: Here, the sound method behaves differently depending on whether the object is a Dog, Cat or Animal and this decision happens at runtime. This dynamic nature makes Python particularly powerful for runtime polymorphism.

Inheritance Class Polymorphism

Inheritance-based polymorphism occurs when a subclass overrides a method from its parent class, providing a specific implementation. This process of re-implementing a method in the child class is known as [Method Overriding](#).

Example:

```
class Animal:
    def sound(self):
        return "Some generic animal sound"
class Dog(Animal):
    def sound(self):
        return "Bark"
class Cat(Animal):
    def sound(self):
        return "Meow"
```

Explanation:

- **Class Animal:** Acts as the base (parent) class. Contains a method sound that provides a default behavior, returning "Some generic animal sound". This serves as a generic representation of the sound method for all animals.
- **Class Dog:** Inherits from the Animal class (denoted by class Dog(Animal)). Overrides the sound method to return "Woof Woof!", a behavior specific to dogs. This demonstrates method overriding, where the subclass modifies the implementation of the parent class's method.

- **Class Cat:** Inherits from the Animal class (denoted by class Cat(Animal)). Overrides the sound method to return "Meow", a behavior specific to cats. Like Dog, this also demonstrates method overriding.

What is method overriding in polymorphism?

Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass. This allows the subclass to customize or completely replace the behavior of the method inherited from the superclass.

Example:

```
class Animal:
    def sound(self):
        return "Some sound"
class Dog(Animal):
    def sound(self):
        return "Bark"
dog = Dog()
print(dog.sound()) # Output: "Bark"
```

How to achieve polymorphism using class inheritance?

Polymorphism can be achieved using class inheritance by defining a common interface in a base class and then providing specific implementations in derived classes. This allows objects of different classes to be treated as objects of a common superclass.

Example:

```
class Shape:
    def draw(self):
        raise NotImplementedError("Subclass must implement abstract method")
class Circle(Shape):
    def draw(self):
        return "Drawing a circle"
class Square(Shape):
    def draw(self):
        return "Drawing a square"
shapes = [Circle(), Square()]
for shape in shapes:
    print(shape.draw())
# Output:
```

```
# Drawing a circle  
# Drawing a square
```

Can polymorphism be achieved with functions in Python?

Yes, polymorphism can be achieved with functions in Python through duck typing, where the method or function operates on any object that supports the required method or attribute, regardless of the object's class.

Example:

```
def make_sound(animal):  
    return animal.sound()  
  
class Dog:  
    def sound(self):  
        return "Bark"  
  
class Cat:  
    def sound(self):  
        return "Meow"  
  
animals = [Dog(), Cat()]  
for animal in animals:  
    print(make_sound(animal))  
  
# Output:  
# Bark  
# Meow
```

How does polymorphism help in writing flexible and maintainable code?

Polymorphism allows you to write more flexible and maintainable code by enabling you to use a single interface to represent different underlying forms (data types). This means you can add new classes with their own implementations without altering the existing code, making it easier to extend and maintain.

Example:

```
class Vehicle:  
    def move(self):  
        raise NotImplementedError("Subclass must implement abstract method")  
  
class Car(Vehicle):  
    def move(self):  
        return "Car is moving"  
  
class Bike(Vehicle):  
    def move(self):
```

```
        return "Bike is moving"
vehicles = [Car(), Bike()]
for vehicle in vehicles:
    print(vehicle.move())
# Output:
# Car is moving
# Bike is moving
```

What is the role of abstract base classes in polymorphism?

Abstract base classes (ABCs) define a common interface for a group of subclasses. They cannot be instantiated themselves and require subclasses to provide implementations for their abstract methods. ABCs ensure that derived classes adhere to a specific protocol, thus supporting polymorphism.

Example:

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
class Dog(Animal):
    def sound(self):
        return "Bark"
class Cat(Animal):
    def sound(self):
        return "Meow"
animals = [Dog(), Cat()]
for animal in animals:
    print(animal.sound())
# Output:
# Bark
# Meow
```

4. Abstract Classes

In Python, an abstract class is a class that cannot be instantiated on its own and is designed to be a blueprint for other classes. Abstract classes allow us to define methods that must be implemented by subclasses, ensuring a consistent interface while still allowing the subclasses to provide specific implementations.

Abstract Base Classes in Python

It defines methods that must be implemented by its subclasses, ensuring that the subclasses follow a consistent structure. ABCs allow you to define common interfaces that various subclasses can implement while enforcing a level of abstraction.

Python provides the abc module to define ABCs and enforce the implementation of abstract methods in subclasses.

Example:

```
from abc import ABC, abstractmethod
# Define an abstract class
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass # This is an abstract method, no implementation here.
# Concrete subclass of Animal
class Dog(Animal):
    def sound(self):
        return "Bark" # Providing the implementation of the abstract method
# Create an instance of Dog
dog = Dog()
print(dog.sound()) # Output: Bark
```

Code Explanation:

- **Abstract Base Class:** Animal is an abstract class that inherits from ABC (Abstract Base Class). This class cannot be instantiated directly because it contains an abstract method sound(). The @abstractmethod decorator is used to mark sound() as an abstract method. This means any subclass must implement this method to be instantiated.
- **Concrete Subclass:** Dog is a subclass of Animal that provides an implementation for the sound() method. This allows the Dog class to be instantiated and used.
- **Instantiation:** We create an instance of Dog and call the sound() method, which returns "Bark".

The main components of an abstract class are:

- [Abstract Base Classes in Python](#)
- [Abstract Methods](#)
- [Concrete Methods](#)
- [Abstract Properties](#)
- [Abstract Class Instantiation](#)

Abstract Methods

Abstract methods are methods that are defined in an abstract class but do not have an implementation. They serve as a blueprint for the subclasses, ensuring that they provide their own implementation.

Example:

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass # Abstract method, no implementation here
```

Explanation:

- make_sound() is an abstract method in the Animal class, so it doesn't have any code inside it.

Concrete Methods

Concrete methods are methods that have full implementations in an abstract class. These methods can be inherited by subclasses and used directly without needing to be redefined.

Example:

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass # Abstract method, to be implemented by subclasses
    def move(self):
        return "Moving" # Concrete method with implementation
```

Explanation:

- The move() method is a concrete method in the Animal class. It is implemented and does not need to be overridden by the Dog class.

Abstract Properties

Abstract properties work like abstract methods but are used for properties. These properties are declared with the `@property` decorator and marked as abstract using `@abstractmethod`. Subclasses must implement these properties.

Example:

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @property
    @abstractmethod
    def species(self):
        pass # Abstract property, must be implemented by subclasses
class Dog(Animal):
    @property
    def species(self):
        return "Canine"
# Instantiate the concrete subclass
dog = Dog()
print(dog.species)
```

Explanation:

- species is an abstract property in the Animal class and it is marked as `@abstractmethod`.
- The Dog class implements the species property, making it a concrete subclass that can be instantiated.
- Abstract properties enforce that a subclass provides the property's implementation.

Abstract Class Instantiation

Abstract classes cannot be instantiated directly. This is because they contain one or more abstract methods or properties that lack implementations. Attempting to instantiate an abstract class results in a `TypeError`.

Example:

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
```

```
# Trying to instantiate the abstract class directly
# This will raise an error:
# animal = Animal() # TypeError: Can't instantiate abstract class Animal with abstract
# methods make_sound
```

Explanation:

- The Animal class is abstract because it has the make_sound() method as an abstract method.
- If you try to instantiate Animal directly (e.g., animal = Animal()), Python raises a TypeError because you can't instantiate a class with unimplemented abstract methods.
- Only subclasses that implement all abstract methods can be instantiated.

What is the Purpose of Abstract Classes in Python?

Abstract classes in Python are used to define a common interface for a group of subclasses. They enforce certain methods to be implemented by any subclass, ensuring a consistent API and encouraging code reuse. Abstract classes themselves cannot be instantiated and are meant to be inherited by other classes that provide concrete implementations of the abstract methods.

How to Define and Use an Abstract Class in Python?

To define an abstract class in Python, you typically use the abc (Abstract Base Class) module. An abstract class is defined by inheriting from ABC and using the @abstractmethod decorator to define abstract methods that must be implemented by subclasses.

Example:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
```

```
class Dog(Animal):
    def sound(self):
        return "Bark"
```

```
dog = Dog()
print(dog.sound()) # Output: Bark
```

What is the 'abc' Module and How is It Used in Python?

The *abc* module in Python provides tools for defining abstract base classes. It includes the *ABC* class and the *abstractmethod* decorator, which are used to create abstract classes and methods.

Using the *abc* Module:

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

```
class Rectangle(Shape):
    def area(self):
        return 10 * 20
```

```
rectangle = Rectangle()
print(rectangle.area()) # Output: 200
```

How to Define Abstract Methods in Python?

Abstract methods in Python are defined using the *@abstractmethod* decorator from the *abc* module. These methods do not provide any implementation in the abstract class and must be overridden in any concrete subclass.

Example:

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
    @abstractmethod
    def start_engine(self):
        pass
```

```
class Car(Vehicle):
    def start_engine(self):
        return "Car engine started"
```

```
car = Car()
print(car.start_engine()) # Output: Car engine started
```

Why Should Abstract Classes Be Used in Python?

Abstract classes should be used in Python for several reasons:

1. **Enforcing a Contract:** *Ensure that all subclasses implement certain methods, providing a consistent interface.*
2. **Code Reusability:** *Define common functionality in the abstract class and reuse it in all subclasses.*
3. **Encouraging Design Principles:** *Promote the use of design patterns and principles like SOLID.*
4. **Reducing Code Duplication:** *Define shared methods and properties once in the abstract class.*
5. **Improved Readability and Maintainability:** *Provide clear guidelines for developers about the structure and behavior expected from subclasses*

5. Exceptional Handling

Python Exception Handling handles errors that occur during the execution of a program. Exception handling allows to respond to the error, instead of crashing the running program. It enables you to catch and manage errors, making your code more robust and user-friendly.

Example: Trying to divide a number by zero will cause an exception.

```
# Example of an exception
n = 10
try:
    res = n / 0 # This will raise a ZeroDivisionError

except ZeroDivisionError:
    print("Can't be divided by zero!")
```

Output

```
Can't be divided by zero!
```

Explanation: In this example, dividing number by 0 raises a [ZeroDivisionError](#). The try block contains the code that might cause an exception and the except block handles the exception, printing an error message instead of stopping the program.

Difference Between Exception and Error

Error: Errors are serious issues that a program should not try to handle. They are usually problems in the code's logic or configuration and need to be fixed by the programmer. Examples include syntax errors and memory errors.

Exception: Exceptions are less severe than errors and can be handled by the program. They occur due to situations like invalid input, missing files or network issues.

Example:

```
# Syntax Error (Error)
print("Hello world" # Missing closing parenthesis
# ZeroDivisionError (Exception)
n = 10
res = n / 0
```

Explanation: A syntax error is a coding mistake that prevents the code from running. In contrast, an exception like ZeroDivisionError can be managed during the program's execution using exception handling.

Syntax and Usage

Exception handling in Python is done using the try, except, else and finally blocks.

try:

Code that might raise an exception

except SomeException:

Code to handle the exception

else:

Code to run if no exception occurs

finally:

Code to run regardless of whether an exception occurs

try, except, else and finally Blocks

try Block: [try block](#) lets us test a block of code for errors. Python will “try” to execute the code in this block. If an exception occurs, execution will immediately jump to the except block.

except Block: [except block](#) enables us to handle the error or exception. If the code inside the try block throws an error, Python jumps to the except block and executes it. We can handle specific exceptions or use a general except to catch all exceptions.

else Block: [else block](#) is optional and if included, must follow all except blocks. The else block runs only if no exceptions are raised in the try block. This is useful for code that should execute if the try block succeeds.

finally Block: [finally block](#) always runs, regardless of whether an exception occurred or not. It is typically used for cleanup operations (closing files, releasing resources).

Example:

```
try:
    n = 0
    res = 100 / n
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("Enter a valid number!")
else:
    print("Result is", res)
finally:
    print("Execution complete.")
```

Output

```
You can't divide by zero!
Execution complete.
```

Explanation:

try block asks for user input and tries to divide 100 by the input number.

except blocks handle ZeroDivisionError and ValueError.

else block runs if no exception occurs, displaying the result.

finally block runs regardless of the outcome, indicating the completion of execution.

Common Exceptions in Python

Python has many [built-in exceptions](#), each representing a specific error condition. Some common ones include:

Exception Name	Description
BaseException	The base class for all built-in exceptions.
Exception	The base class for all non-exit exceptions.
ArithmeticError	Base class for all errors related to arithmetic operations.

Exception Name	Description
<u>ZeroDivisionError</u>	Raised when a division or modulo operation is performed with zero as the divisor.
<u>OverflowError</u>	Raised when a numerical operation exceeds the maximum limit of a data type.
<u>FloatingPointError</u>	Raised when a floating-point operation fails.
<u>AssertionError</u>	Raised when an assert statement fails.
<u>AttributeError</u>	Raised when an attribute reference or assignment fails.
<u>IndexError</u>	Raised when a sequence subscript is out of range.
<u>KeyError</u>	Raised when a dictionary key is not found.
<u>MemoryError</u>	Raised when an operation runs out of memory.
<u>NameError</u>	Raised when a local or global name is not found.
<u>OSError</u>	Raised when a system-related operation (like file I/O) fails.
<u>TypeError</u>	Raised when an operation or function is applied to an object of inappropriate type.
<u>ValueError</u>	Raised when a function receives an argument of the right type but inappropriate value.
<u>ImportError</u>	Raised when an import statement has issues.

Exception Name	Description
<u>ModuleNotFoundError</u>	Raised when a module cannot be found.

Python Catching Exceptions

When working with exceptions in Python, we can handle errors more efficiently by specifying the types of exceptions we expect. This can make code both safer and easier to debug.

Catching Specific Exceptions

Catching specific exceptions makes code to respond to different exception types differently.

Example:

```
try:
    x = int("str") # This will cause ValueError
    #inverse
    inv = 1 / x
except ValueError:
    print("Not Valid!")
except ZeroDivisionError:
    print("Zero has no inverse!")
```

Output

```
Not Valid!
```

Explanation:

The ValueError is caught because the string "str" cannot be converted to an integer.

If x were 0 and conversion successful, the ZeroDivisionError would be caught when attempting to calculate its inverse.

Catching Multiple Exceptions

We can catch multiple exceptions in a single block if we need to handle them in the same way or we can separate them if different types of exceptions require different handling.

Example:

```
a = ["10", "twenty", 30] # Mixed list of integers and strings
try:
    total = int(a[0]) + int(a[1]) # 'twenty' cannot be converted to int
except (ValueError, TypeError) as e:
```

```
print("Error", e)
except IndexError:
    print("Index out of range.")
```

Output

Error invalid literal for int() with base 10: 'twenty'

Explanation:

- The ValueError is caught when trying to convert “twenty” to an integer.
- TypeError might occur if the operation was incorrectly applied to non-integer types, but it’s not triggered in this specific setup.
- IndexError would be caught if an index outside the range of the list was accessed, but in this scenario, it’s under control.

Catch-All Handlers and Their Risks

Here’s a simple calculation that may fail due to various reasons.

Example:

```
try:
    # Simulate risky calculation: incorrect type operation
    res = "100" / 20
except ArithmeticError:
    print("Arithmetic problem.")
except:
    print("Something went wrong!")
```

Output

Something went wrong!

Explanation:

An ArithmeticError (more specific like ZeroDivisionError) might be caught if this were a number-to-number division error. However, TypeError is actually triggered here due to attempting to divide a string by a number.

catch-all except: is used to catch the TypeError, demonstrating the risk that the programmer might not realize the actual cause of the error (type mismatch) without more detailed error logging.

Raise an Exception

We [raise](#) an exception in Python using the raise keyword followed by an instance of the exception class that we want to trigger. We can choose from built-in exceptions or define our own custom exceptions by inheriting from Python’s built-in Exception class.

Basic Syntax:

raise ExceptionType("Error message")

Example:

```
def set(age):  
    if age < 0:  
        raise ValueError("Age cannot be negative.")  
    print(f"Age set to {age}")  
try:  
    set(-5)  
except ValueError as e:  
    print(e)
```

Output

Age cannot be negative.

Explanation:

The function set checks if the age is negative. If so, it raises a ValueError with a message explaining the issue.

This ensures that the age attribute cannot be set to an invalid state, thus maintaining the integrity of the data.

Advantages of Exception Handling:

Improved program reliability: By handling exceptions properly, you can prevent your program from crashing or producing incorrect results due to unexpected errors or input.

Simplified error handling: Exception handling allows you to separate error handling code from the main program logic, making it easier to read and maintain your code.

Cleaner code: With exception handling, you can avoid using complex conditional statements to check for errors, leading to cleaner and more readable code.

Easier debugging: When an exception is raised, the Python interpreter prints a traceback that shows the exact location where the exception occurred, making it easier to debug your code.

Disadvantages of Exception Handling:

Performance overhead: Exception handling can be slower than using conditional statements to check for errors, as the interpreter has to perform additional work to catch and handle the exception.

Increased code complexity: Exception handling can make your code more complex, especially if you have to handle multiple types of exceptions or implement complex error handling logic.

Possible security risks: Improperly handled exceptions can potentially reveal sensitive information or create security vulnerabilities in your code, so it's important to handle exceptions carefully and avoid exposing too much information about your program.

6. Regular Expressions using “re” module.

A **Regular Expression or RegEx** is a special sequence of characters that uses a search pattern to find a string or set of strings.

It can detect the presence or absence of a text by matching it with a particular pattern and also can split a pattern into one or more sub-patterns.

Regex Module in Python

[Python](#) has a built-in module named “re” that is used for regular expressions in Python.

We can import this module by using the [import statement](#).

Example: Importing re module in Python

```
# importing re module
```

```
import re
```

How to Use RegEx in Python?

You can use RegEx in Python after importing re module.

Example:

This Python code uses regular expressions to search for the word “portal” in the given string and then prints the start and end indices of the matched word within the string.

```
import re
```

```
s = 'GeeksforGeeks: A computer science portal for geeks'
```

```
match = re.search(r'portal', s)
```

```
print('Start Index:', match.start())
```

```
print('End Index:', match.end())
```

Output

```
Start Index: 34
```

```
End Index: 40
```

Note: Here r character (r'portal') stands for raw, not regex. The raw string is slightly different from a regular string, it won't interpret the \ character as an escape character. This is because the regular expression engine uses \ character for its own escaping purpose.

Before starting with the Python regex module let's see how to actually write regex using metacharacters or special sequences.

RegEx Functions

`re` module contains many [functions](#) that help us to search a string for a match. Let's see various functions provided by this module to work with regex in Python.

Function	Description
<code>re.findall()</code>	finds and returns all matching occurrences in a list
<code>re.compile()</code>	Regular expressions are compiled into pattern objects
<code>re.split()</code>	Split string by the occurrences of a character or a pattern.
<code>re.sub()</code>	Replaces all occurrences of a character or pattern with a replacement string.
<code>re.escape()</code>	Escapes special character
<code>re.search()</code>	Searches for first occurrence of character or pattern

Let's see the working of these RegEx functions with definition and examples:

1. `re.findall()`

Return all non-overlapping matches of pattern in string, as a list of strings. The string is scanned left-to-right, and matches are returned in the order found.

Finding all occurrences of a pattern

This code uses a regular expression (`\d+`) to find all the sequences of one or more digits in the given string. It searches for numeric values and stores them in a list. In this example, it finds and prints the numbers **"123456789"** and **"987654321"** from the input string.

```
import re
string = """Hello my Number is 123456789 and
          my friend's number is 987654321"""
regex = '\d+'
match = re.findall(regex, string)
print(match)
```

Output

```
['123456789', '987654321']
```

2. re.compile()

Regular expressions are compiled into pattern objects, which have methods for various operations such as searching for pattern matches or performing string substitutions.

Example 1:

The code uses a regular expression pattern **[a-e]** to find and list all lowercase letters from 'a' to 'e' in the input string **"Aye, said Mr. Gibenson Stark"**. The output will be **['e', 'a', 'd', 'b', 'e']**, which are the matching characters.

```
import re
p = re.compile('[a-e]')
print(p.findall("Aye, said Mr. Gibenson Stark"))
```

Output

```
['e', 'a', 'd', 'b', 'e', 'a']
```

Understanding the Output:

- First occurrence is 'e' in "Aye" and not 'A', as it is Case Sensitive.
- Next Occurrence is 'a' in "said", then 'd' in "said", followed by 'b' and 'e' in "Gibenson", the Last 'a' matches with "Stark".
- Metacharacter backslash '\' has a very important role as it signals various sequences. If the backslash is to be used without its special meaning as metacharacter, use '\\'

Example 2: Set class **[\s,.]** will match any whitespace character, ',', or '.'.

The code uses regular expressions to find and list all single digits and sequences of digits in the given input strings. It finds single digits with **\d** and sequences of digits with **\d+**.

```
import re
p = re.compile("\d")
print(p.findall("I went to him at 11 A.M. on 4th July 1886"))
p = re.compile("\d+")
print(p.findall("I went to him at 11 A.M. on 4th July 1886"))
```

Output

```
['1', '1', '4', '1', '8', '8', '6']
['11', '4', '1886']
```

Example 3:

The code uses regular expressions to find and list word characters, sequences of word characters, and non-word characters in input strings. It provides lists of the matched characters or sequences.

```
import re
p = re.compile('\w')
print(p.findall("He said * in some_lang."))
p = re.compile('\w+')
print(p.findall("I went to him at 11 A.M., he \
said *** in some_language."))
p = re.compile('\W')
print(p.findall("he said *** in some_language."))
```

Output

```
['H', 'e', 's', 'a', 'l', 'd', 'i', 'n', 's', 'o', 'm', 'e', '_', 'l', 'a', 'n', 'g']
['I', 'went', 'to', 'him', 'at', '11', 'A', 'M', 'he', 'said', 'in', 'some_language']
[' ', '*', '*', '*', ' ', ' ', ' ']
```

Example 4:

The code uses a regular expression pattern 'ab*' to find and list all occurrences of 'ab' followed by zero or more 'b' characters in the input string "ababbaabbb". It returns the following list of matches: ['ab', 'abb', 'abbb'].

```
import re
p = re.compile('ab*')
print(p.findall("ababbaabbb"))
```

Output

```
['ab', 'abb', 'a', 'abbb']
```

Understanding the Output:

- Our RE is ab*, which 'a' accompanied by any no. of 'b's, starting from 0.
- Output 'ab', is valid because of single 'a' accompanied by single 'b'.
- Output 'abb', is valid because of single 'a' accompanied by 2 'b'.
- Output 'a', is valid because of single 'a' accompanied by 0 'b'.
- Output 'abbb', is valid because of single 'a' accompanied by 3 'b'.

3. re.split()

Split string by the occurrences of a character or a pattern, upon finding that pattern, the remaining characters from the string are returned as part of the resulting list.

Syntax :

```
re.split(pattern, string, maxsplit=0, flags=0)
```

The First parameter, pattern denotes the regular expression, string is the given string in which pattern will be searched for and in which splitting occurs, maxsplit if not

provided is considered to be zero '0', and if any nonzero value is provided, then at most that many splits occur. If maxsplit = 1, then the string will split once only, resulting in a list of length 2. The flags are very useful and can help to shorten code, they are not necessary parameters, eg: flags = re.IGNORECASE, in this split, the case, i.e. the lowercase or the uppercase will be ignored.

Example 1:

Splits a string using non-word characters and spaces as delimiters, returning words: ['Words', 'words', 'Words']. Considers apostrophes as non-word characters: ['Word', 's', 'words', 'Words']. Splits using non-word characters and digits: ['On', '12th', 'Jan', '2016', 'at', '11', '02', 'AM']. Splits using digits as the delimiter: ['On ', 'th Jan ', 'at ', ':', 'AM'].

```
from re import split
print(split('\W+', 'Words, words , Words'))
print(split('\W+', "Word's words Words"))
print(split('\W+', 'On 12th Jan 2016, at 11:02 AM'))
print(split('\d+', 'On 12th Jan 2016, at 11:02 AM'))
```

Output

```
['Words', 'words', 'Words']
['Word', 's', 'words', 'Words']
['On', '12th', 'Jan', '2016', 'at', '11', '02', 'AM']
['On ', 'th Jan ', 'at ', ':', 'AM']
```

Example 2:

First statement splits the string at the first occurrence of one or more digits: ['On ', 'th Jan 2016, at 11:02 AM']. second splits the string using lowercase letters a to f as delimiters, case-insensitive: [' ', 'y, ', 'oy oh ', 'oy, ', 'ome here']. Third splits the string using lowercase letters a to f as delimiters, case-sensitive: [' ', 'ey, Boy oh ', 'oy, ', 'ome here'].

```
import re
print(re.split('\d+', 'On 12th Jan 2016, at 11:02 AM', 1))
print(re.split('[a-f]+', 'Aey, Boy oh boy, come here', flags=re.IGNORECASE))
print(re.split('[a-f]+', 'Aey, Boy oh boy, come here'))
```

Output

```
['On ', 'th Jan 2016, at 11:02 AM']
[' ', 'y, ', 'oy oh ', 'oy, ', 'om', ' h', 'r', '']
['A', 'y, Boy oh ', 'oy, ', 'om', ' h', 'r', '']
```

4. re.sub()

The 'sub' in the function stands for SubString, a certain regular expression pattern is searched in the given string(3rd parameter), and upon finding the substring pattern is replaced by repl(2nd parameter), count checks and maintains the number of times this occurs.

Syntax:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

Example 1:

- First statement replaces all occurrences of 'ub' with '_*' (case-insensitive): **'S-*ject has -*er booked already'** .
- Second statement replaces all occurrences of 'ub' with '_*' (case-sensitive): **'S-*ject has Uber booked already'** .
- Third statement replaces the first occurrence of 'ub' with '_*' (case-insensitive): **'S-*ject has Uber booked already'** .
- Fourth replaces 'AND' with ' & ' (case-insensitive): **'Baked Beans & Spam'** .

```
import re
print(re.sub('ub', '_*', 'Subject has Uber booked already',
            flags=re.IGNORECASE))
print(re.sub('ub', '_*', 'Subject has Uber booked already'))
print(re.sub('ub', '_*', 'Subject has Uber booked already',
            count=1, flags=re.IGNORECASE))
print(re.sub(r'\sAND\s', ' & ', 'Baked Beans And Spam',
            flags=re.IGNORECASE))
```

Output

```
S-*ject has -*er booked already
S-*ject has Uber booked already
S-*ject has Uber booked already
Baked Beans & Spam
```

5. re.subn()

subn() is similar to sub() in all ways, except in its way of providing output. It returns a tuple with a count of the total of replacement and the new string rather than just the string.

Syntax:

```
re.subn(pattern, repl, string, count=0, flags=0)
```

Example:

re.subn() replaces all occurrences of a pattern in a string and returns a tuple with the

modified string and the count of substitutions made. It's useful for both case-sensitive and case-insensitive substitutions.

```
import re
print(re.subn('ub', '-*', 'Subject has Uber booked already'))
t = re.subn('ub', '-*', 'Subject has Uber booked already',
            flags=re.IGNORECASE)
print(t)
print(len(t))
print(t[0])
```

Output

```
('S-*ject has Uber booked already', 1)
('S-*ject has -*er booked already', 2)
2
S-*ject has -*er booked already
```

6. re.escape()

Returns string with all non-alphanumerics backslashed, this is useful if you want to match an arbitrary literal string that may have regular expression metacharacters in it.

Syntax:

```
re.escape(string)
```

Example:

re.escape() is used to escape special characters in a string, making it safe to be used as a pattern in regular expressions. It ensures that any characters with special meanings in regular expressions are treated as literal characters.

```
import re
print(re.escape("This is Awesome even 1 AM"))
print(re.escape("I Asked what is this [a-9], he said \t `WoW"))
```

Output

```
This\ is\ Awesome\ even\ 1\ AM
I\ Asked\ what\ is\ this\ \[a-9]\,\ he\ said\ \ \ \ `WoW
```

7. re.search()

This method either returns None (if the pattern doesn't match), or a re.MatchObject contains information about the matching part of the string. This method stops after the first match, so this is best suited for testing a regular expression more than extracting data.

Example: Searching for an occurrence of the pattern

This code uses a regular expression to search for a pattern in the given string. If a match is found, it extracts and prints the matched portions of the string.

In this specific example, it searches for a pattern that consists of a month (letters) followed by a day (digits) in the input string "I was born on June 24". If a match is found, it prints the full match, the month, and the day.

```
import re
regex = r"([a-zA-Z]+) (\d+)"
match = re.search(regex, "I was born on June 24")
if match != None:
    print ("Match at index %s, %s" % (match.start(), match.end()))
    print ("Full match: %s" % (match.group(0)))
    print ("Month: %s" % (match.group(1)))
    print ("Day: %s" % (match.group(2)))
else:
    print ("The regex pattern does not match.")
```

Output

```
Match at index 14, 21
Full match: June 24
Month: June
Day: 24
```

Meta-characters

Metacharacters are the characters with special meaning.

To understand the RE analogy, Metacharacters are useful and important. They will be used in functions of module re. Below is the list of metacharacters.

MetaCharacters	Description
\	Used to drop the special meaning of character following it
[]	Represent a character class
^	Matches the beginning
\$	Matches the end

MetaCharacters	Description
.	Matches any character except newline
	Means OR (Matches with any of the characters separated by it.
?	Matches zero or one occurrence
*	Any number of occurrences (including 0 occurrences)
+	One or more occurrences
{ }	Indicate the number of occurrences of a preceding regex to match.
()	Enclose a group of Regex

Let's discuss each of these metacharacters in detail:

1. \ – Backslash

The backslash (\) makes sure that the character is not treated in a special way. This can be considered a way of escaping metacharacters.

For example, if you want to search for the dot(.) in the string then you will find that dot(.) will be treated as a special character as is one of the metacharacters (as shown in the above table). So for this case, we will use the backslash(\) just before the dot(.) so that it will lose its specialty. See the below example for a better understanding.

Example:

The first search (**re.search(r".", s)**) matches any character, not just the period, while the second search (**re.search(r'\.', s)**) specifically looks for and matches the period character.

```
import re
s = 'geeks.forgeeks'
# without using \
match = re.search(r".", s)
print(match)
```

```
# using \
match = re.search(r'\.', s)
print(match)
```

Output

```
<re.Match object; span=(0, 1), match='g'>
<re.Match object; span=(5, 6), match='.'>
```

2. [] – Square Brackets

Square Brackets ([]) represent a character class consisting of a set of characters that we wish to match. For example, the character class [abc] will match any single a, b, or c.

We can also specify a range of characters using – inside the square brackets. For example,

- [0, 3] is same as [0123]
- [a-c] is same as [abc]
 - o We can also invert the character class using the caret(^) symbol. For example,
- [^0-3] means any character except 0, 1, 2, or 3
- [^a-c] means any character except a, b, or c

Example:

In this code, you're using regular expressions to find all the characters in the string that fall within the range of 'a' to 'm'. The **re.findall()** function returns a list of all such characters. In the given string, the characters that match this pattern are: 'c', 'k', 'b', 'f', 'j', 'e', 'h', 'l', 'd', 'g'.

```
import re
string = "The quick brown fox jumps over the lazy dog"
pattern = "[a-m]"
result = re.findall(pattern, string)
print(result)
```

Output

```
['h', 'e', 'l', 'c', 'k', 'b', 'f', 'j', 'm', 'e', 'h', 'e', 'l', 'a', 'd', 'g']
```

3. ^ – Caret

Caret (^) symbol matches the beginning of the string i.e. checks whether the string starts with the given character(s) or not. For example –

- ^g will check if the string starts with g such as geeks, globe, girl, g, etc.
- ^ge will check if the string starts with ge such as geeks, geeksforgeeks, etc.

Example:

This code uses regular expressions to check if a list of strings starts with **“The”**. If a string begins with **“The,”** it's marked as **“Matched”** otherwise, it's labeled as **“Not matched”**.

```
import re
regex = r'^The'
strings = ['The quick brown fox', 'The lazy dog', 'A quick brown fox']
for string in strings:
    if re.match(regex, string):
        print(f'Matched: {string}')
    else:
        print(f'Not matched: {string}')
```

Output

```
Matched: The quick brown fox
Matched: The lazy dog
Not matched: A quick brown fox
```

4. \$ – Dollar

Dollar(\$) symbol matches the end of the string i.e checks whether the string ends with the given character(s) or not. For example–

- s\$ will check for the string that ends with a such as geeks, ends, s, etc.
- ks\$ will check for the string that ends with ks such as geeks, geeksforgeeks, ks, etc.

Example:

This code uses a regular expression to check if the string ends with **“World!”**. If a match is found, it prints **“Match found!”** otherwise, it prints **“Match not found”**.

```
import re
string = "Hello World!"
pattern = r"World!$"
match = re.search(pattern, string)
if match:
    print("Match found!")
else:
    print("Match not found.")
```

Output

```
Match found!
```

5. . – Dot

Dot(.) symbol matches only a single character except for the newline character (\n). For example –

- a.b will check for the string that contains any character at the place of the dot such as acb, acbd, abbb, etc
- .. will check if the string contains at least 2 characters

Example:

This code uses a regular expression to search for the pattern “**brown.fox**” within the string. The dot (.) in the pattern represents any character. If a match is found, it prints “**Match found!**” otherwise, it prints “**Match not found**”.

```
import re
string = "The quick brown fox jumps over the lazy dog."
pattern = r"brown.fox"
match = re.search(pattern, string)
if match:
    print("Match found!")
else:
    print("Match not found.")
```

Output

```
Match found!
```

6. | – Or

Or symbol works as the or operator meaning it checks whether the pattern before or after the or symbol is present in the string or not. For example –

- a|b will match any string that contains a or b such as acd, bcd, abcd, etc.

7. ? – Question Mark

The question mark (?) is a quantifier in regular expressions that indicates that the preceding element should be matched zero or one time. It allows you to specify that the element is optional, meaning it may occur once or not at all. For example,

- ab?c will be matched for the string ac, acb, dabc but will not be matched for abbc because there are two b. Similarly, it will not be matched for abdc because b is not followed by c.

8.* – Star

Star (*) symbol matches zero or more occurrences of the regex preceding the * symbol. For example –

- ab*c will be matched for the string ac, abc, abbbc, dabc, etc. but will not be matched for abdc because b is not followed by c.

9. + – Plus

Plus (+) symbol matches one or more occurrences of the regex preceding the + symbol. For example –

- `ab+c` will be matched for the string `abc`, `abbc`, `dabc`, but will not be matched for `ac`, `abdc`, because there is no `b` in `ac` and `b`, is not followed by `c` in `abdc`.

10. {m, n} – Braces

Braces match any repetitions preceding regex from `m` to `n` both inclusive. For example –

- `a{2, 4}` will be matched for the string `aaab`, `baaaac`, `gaad`, but will not be matched for strings like `abc`, `bc` because there is only one `a` or no `a` in both the cases.

11. (<regex>) – Group

Group symbol is used to group sub-patterns. For example –

- `(a|b)cd` will match for strings like `acd`, `abcd`, `gacd`, etc.

Special Sequences

Special sequences do not match for the actual character in the string instead it tells the specific location in the search string where the match must occur. It makes it easier to write commonly used patterns.

List of special sequences

Special Sequence	Description	Examples	
\A	Matches if the string begins with the given character	\Afor	for geeks
			for the world

Special Sequence	Description	Examples	
\b	Matches if the word begins or ends with the given character. \b(string) will check for the beginning of the word and (string)\b will check for the ending of the word.	\bge	geeks
			get
\B	It is the opposite of the \b i.e. the string should not start or end with the given regex.	\Bge	together
			forge
\d	Matches any decimal digit, this is equivalent to the set class [0-9]	\d	123
			gee1
\D	Matches any non-digit character, this is equivalent to the set class [^0-9]	\D	geeks
			geek1

Special Sequence	Description	Examples	
\s	Matches any whitespace character.	\s	gee ks
			a bc a
\S	Matches any non-whitespace character	\S	a bd
			abcd
\w	Matches any alphanumeric character, this is equivalent to the class [a-zA-Z0-9_].	\w	123
			geeKs4
\W	Matches any non-alphanumeric character.	\W	>\$
			gee<>
\Z	Matches if the string ends with the given regex	ab\Z	abcdab
			abababab

Sets for character matching

A **Set** is a set of characters enclosed in '[']' brackets. Sets are used to match a single character in the set of characters specified between brackets. Below is the list of Sets:

Set	Description
\{n,\}	Quantifies the preceding character or group and matches at least n occurrences.

Set	Description
*	Quantifies the preceding character or group and matches zero or more occurrences.
[0123]	Matches the specified digits (0, 1, 2, or 3)
[^arn]	matches for any character EXCEPT a, r, and n
\d	Matches any digit (0-9).
[0-5][0-9]	matches for any two-digit numbers from 00 and 59
\w	Matches any alphanumeric character (a-z, A-Z, 0-9, or _).
[a-n]	Matches any lower case alphabet between a and n.
\D	Matches any non-digit character.
[arn]	matches where one of the specified characters (a, r, or n) are present
[a-zA-Z]	matches any character between a and z, lower case OR upper case
[0-9]	matches any digit between 0 and 9

Match Object

A Match object contains all the information about the search and the result and if there is no match found then None will be returned. Let's see some of the commonly used methods and attributes of the match object.

1. Getting the string and the regex

match.re attribute returns the regular expression passed and **match.string** attribute

returns the string passed.

Example: Getting the string and the regex of the matched object

The code searches for the letter “G” at a word boundary in the string “Welcome to GeeksForGeeks” and prints the regular expression pattern (**res.re**) and the original string (**res.string**).

```
import re
s = "Welcome to GeeksForGeeks"
res = re.search(r"\bG", s)
print(res.re)
print(res.string)
```

Output

```
re.compile('\\bG')
Welcome to GeeksForGeeks
```

2. Getting index of matched object

- start() method returns the starting index of the matched substring
- end() method returns the ending index of the matched substring
- span() method returns a tuple containing the starting and the ending index of the matched substring

Example: Getting index of matched object

The code searches for the substring “Gee” at a word boundary in the string “Welcome to GeeksForGeeks” and prints the start index of the match (**res.start()**), the end index of the match (**res.end()**), and the span of the match (**res.span()**).

```
import re
s = "Welcome to GeeksForGeeks"
res = re.search(r"\bGee", s)
print(res.start())
print(res.end())
print(res.span())
```

Output

```
11
14
(11, 14)
```

3. Getting matched substring

group() method returns the part of the string for which the patterns match. See the below example for a better understanding.

Example: Getting matched substring

The code searches for a sequence of two non-digit characters followed by a space and the letter 't' in the string "Welcome to GeeksForGeeks" and prints the matched text using `res.group()`.

```
import re
s = "Welcome to GeeksForGeeks"
res = re.search(r"\D{2} t", s)
print(res.group())
```

Output

```
me t
```

In the above example, our pattern specifies for the string that contains at least 2 characters which are followed by a space, and that space is followed by a t.